Solving Systems of Linear Equations, Linear Transformations, and Matrix Composition for Machine Learning and Deep Learning

Comprehensive Mathematical Foundation

December 23, 2025

Contents

1 Solving Systems of Linear Equations **3**
- 1.1 Introduction and Motivation . . . 3
- 1.2 Fundamental Concepts . . . 3
- 1.3 Types of Solutions . . . 3
- 1.4 Methods for Solving Linear Systems . . . 3
  - 1.4.1 Direct Methods . . . 3
  - 1.4.2 Inverse Matrix Method . . . 6
- 1.5 Machine Learning Application: Linear Regression . . . 7
- 1.6 Deep Learning Application: Finding Optimal Weights . . . 9
- 1.7 Computational Complexity Analysis . . . 10

2 Linear Transformations **11**
- 2.1 Introduction and Motivation . . . 11
- 2.2 Fundamental Definitions . . . 11
- 2.3 Properties of Linear Transformations . . . 11
- 2.4 Types of Linear Transformations in 2D and 3D . . . 11
  - 2.4.1 Scaling . . . 11
  - 2.4.2 Rotation . . . 12
  - 2.4.3 Reflection . . . 13
  - 2.4.4 Shear . . . 13
- 2.5 Projection . . . 13
- 2.6 Kernel and Image . . . 14
- 2.7 Machine Learning Applications . . . 15

3 3D Linear Transformations **17**
- 3.1 Introduction . . . 17
- 3.2 Basic 3D Transformations . . . 17
  - 3.2.1 3D Rotation Matrices . . . 17
  - 3.2.2 3D Scaling . . . 18
- 3.3 Computer Vision Application . . . 19

4 Matrix Multiplication as Composition **20**
4.1 Introduction and Significance 20
4.2 Composition of Linear Transformations 20
4.3 Deep Learning Application: Multi-Layer Neural Networks 21
4.4 Practical Implications 24
4.5 Computational Efficiency 24

5 Summary and Connections **25**
5.1 Practical Checklist for ML/DL Practitioners 25

6 Assessment Questions **27**
6.1 Level 1: Basic Understanding 27
6.2 Level 2: Intermediate 27
6.3 Level 3: Advanced 28
6.4 Multiple Choice Questions 29
6.5 Programming Exercises 29

7 Solutions to Assessment Questions **31**
7.1 Level 1 Solutions 31
7.2 Level 2 Solutions 32
7.3 Level 3 Solutions 35
7.4 Multiple Choice Solutions 40

8 Practice Problems **42**
8.1 Computational Problems 42
8.2 Conceptual Problems 42
8.3 Application Problems 42

1 Solving Systems of Linear Equations

1.1 Introduction and Motivation

Why Study Systems of Linear Equations in ML/DL?

Many machine learning algorithms, including linear regression, logistic regression, and neural network training, essentially reduce to solving systems of linear equations. In deep learning, backpropagation can be viewed as a process of solving systems of equations to find optimal parameters.

1.2 Fundamental Concepts

Definition 1.1 (System of Linear Equations)**.** A system of m linear equations in n unknowns is a collection of equations of the form:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2 \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n &= b_m \end{aligned}$$

where a_{ij} are coefficients, x_j are unknowns, and b_i are constants.

Definition 1.2 (Matrix Form)**.** The system can be written compactly as:

$$\mathbf{Ax} = \mathbf{b}$$

where:

- $\mathbf{A} \in \mathbb{R}^{m \times n}$ is the coefficient matrix
- $\mathbf{x} \in \mathbb{R}^n$ is the vector of unknowns
- $\mathbf{b} \in \mathbb{R}^m$ is the constant vector

1.3 Types of Solutions

A system of linear equations can have:

1. **Unique Solution**: Exactly one solution exists
2. **Infinite Solutions**: Infinitely many solutions exist
3. **No Solution**: The system is inconsistent

Theorem 1.1 (Existence and Uniqueness)**.** For a system $\mathbf{Ax} = \mathbf{b}$:

- The system has at least one solution if and only if $\text{rank}(\mathbf{A}) = \text{rank}([\mathbf{A}|\mathbf{b}])$
- The solution is unique if and only if $\text{rank}(\mathbf{A}) = n$ (number of unknowns)

1.4 Methods for Solving Linear Systems

1.4.1 Direct Methods

Gaussian Elimination

Algorithm: Gaussian Elimination

Forward Elimination:

1. Form the augmented matrix $[\mathbf{A}|\mathbf{b}]$
2. For each column k from 1 to n:
   - Choose a pivot element (usually a_{kk})
   - Eliminate all elements below the pivot
3. Result: Upper triangular form

Back Substitution:

1. Start from the last equation
2. Solve for x_n, then x_{n-1}, etc.

Example 1.1 (Gaussian Elimination - Simple Case)**.** Solve the system:

$$\begin{aligned} 2x_1 + x_2 - x_3 &= 8 \\ -3x_1 - x_2 + 2x_3 &= -11 \\ -2x_1 + x_2 + 2x_3 &= -3 \end{aligned}$$

Step 1: Form Augmented Matrix

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ -3 & -1 & 2 & -11 \\ -2 & 1 & 2 & -3 \end{array}\right]$$

Step 2: Forward Elimination
Eliminate x_1 from rows 2 and 3:

- $R_2 \leftarrow R_2 + \frac{3}{2}R_1$:

$$R_2 = [-3, -1, 2, -11] + \frac{3}{2}[2, 1, -1, 8] = [0, 0.5, 0.5, 1]$$

- $R_3 \leftarrow R_3 + R_1$:

$$R_3 = [-2, 1, 2, -3] + [2, 1, -1, 8] = [0, 2, 1, 5]$$

Matrix becomes:

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & 0.5 & 0.5 & 1 \\ 0 & 2 & 1 & 5 \end{array}\right]$$

Eliminate x_2 from row 3:

- $R_3 \leftarrow R_3 - 4R_2$:

$$R_3 = [0, 2, 1, 5] - 4[0, 0.5, 0.5, 1] = [0, 0, -1, 1]$$

Upper triangular form:

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & 0.5 & 0.5 & 1 \\ 0 & 0 & -1 & 1 \end{array}\right]$$

Step 3: Back Substitution
From row 3: $-x_3 = 1 \Rightarrow x_3 = -1$
From row 2: $0.5x_2 + 0.5(-1) = 1 \Rightarrow x_2 = 3$
From row 1: $2x_1 + 3 - (-1) = 8 \Rightarrow x_1 = 2$
Solution: $\mathbf{x} = [2, 3, -1]^T$
Verification:

$$\begin{aligned} 2(2) + 3 - (-1) &= 4 + 3 + 1 = 8\checkmark \\ -3(2) - 3 + 2(-1) &= -6 - 3 - 2 = -11\checkmark \\ -2(2) + 3 + 2(-1) &= -4 + 3 - 2 = -3\checkmark \end{aligned}$$

LU Decomposition

Definition 1.3 (LU Decomposition)**.** For a matrix $\mathbf{A}$, LU decomposition expresses it as:

$$\mathbf{A} = \mathbf{LU}$$

where:

- $\mathbf{L}$ is a lower triangular matrix with 1's on the diagonal
- $\mathbf{U}$ is an upper triangular matrix

Solving $\mathbf{Ax} = \mathbf{b}$ using LU Decomposition:

1. Decompose: $\mathbf{A} = \mathbf{LU}$
2. Solve $\mathbf{Ly} = \mathbf{b}$ for $\mathbf{y}$ (forward substitution)
3. Solve $\mathbf{Ux} = \mathbf{y}$ for $\mathbf{x}$ (backward substitution)

Example 1.2 (LU Decomposition)**.** Decompose and solve:

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 2 \\ 8 \\ 34 \end{bmatrix}$$

Step 1: Find L and U
Using Gaussian elimination with record of multipliers:
First column:

- $l_{21} = \frac{4}{2} = 2$
- $l_{31} = \frac{8}{2} = 4$

After first elimination:

$$\begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 3 & 5 \end{bmatrix}$$

Second column:

- $l_{32} = \frac{3}{1} = 3$

Final upper triangular:

$$\mathbf{U} = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{bmatrix}$$

Lower triangular with multipliers:

$$\mathbf{L} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{bmatrix}$$

Verification:

$$\mathbf{LU} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{bmatrix} = \mathbf{A}\checkmark$$

Step 2: Solve $\mathbf{Ly} = \mathbf{b}$

$$\begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 8 \\ 34 \end{bmatrix}$$

Forward substitution:

$$\begin{aligned} y_1 &= 2 \\ 2y_1 + y_2 = 8 &\Rightarrow y_2 = 8 - 4 = 4 \\ 4y_1 + 3y_2 + y_3 = 34 &\Rightarrow y_3 = 34 - 8 - 12 = 14 \end{aligned}$$

So $\mathbf{y} = [2, 4, 14]^T$

Step 3: Solve $\mathbf{Ux} = \mathbf{y}$

$$\begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \\ 14 \end{bmatrix}$$

Backward substitution:

$$\begin{aligned} 2x_3 = 14 &\Rightarrow x_3 = 7 \\ x_2 + x_3 = 4 &\Rightarrow x_2 = 4 - 7 = -3 \\ 2x_1 + x_2 + x_3 = 2 &\Rightarrow 2x_1 = 2 + 3 - 7 = -2 \Rightarrow x_1 = -1 \end{aligned}$$

Solution: $\mathbf{x} = [-1, -3, 7]^T$

1.4.2 Inverse Matrix Method

Definition 1.4 (Solution via Matrix Inverse)**.** If $\mathbf{A}$ is square and invertible, the solution is:

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$$

Property 1.1 (When to Use Inverse Method)**.** ✓ Good when solving multiple systems with same $\mathbf{A}$ but different $\mathbf{b}$

× Computationally expensive: $O(n^3)$ for inversion

× Numerically unstable for large or ill-conditioned matrices

✓ Used in theoretical derivations

Example 1.3 (Matrix Inverse Method - 2×2 System)**.** Solve:

$$\begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 11 \\ 13 \end{bmatrix}$$

Step 1: Find $\mathbf{A}^{-1}$
For a 2×2 matrix:

$$\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})} \begin{bmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{bmatrix}$$

Calculate determinant:

$$\det(\mathbf{A}) = 3 \cdot 4 - 2 \cdot 1 = 12 - 2 = 10$$

Therefore:

$$\mathbf{A}^{-1} = \frac{1}{10} \begin{bmatrix} 4 & -2 \\ -1 & 3 \end{bmatrix} = \begin{bmatrix} 0.4 & -0.2 \\ -0.1 & 0.3 \end{bmatrix}$$

Step 2: Compute $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$

$$\mathbf{x} = \begin{bmatrix} 0.4 & -0.2 \\ -0.1 & 0.3 \end{bmatrix} \begin{bmatrix} 11 \\ 13 \end{bmatrix} = \begin{bmatrix} 0.4(11) + (-0.2)(13) \\ -0.1(11) + 0.3(13) \end{bmatrix} = \begin{bmatrix} 4.4 - 2.6 \\ -1.1 + 3.9 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 2.8 \end{bmatrix}$$

Solution: $x_1 = 1.8$, $x_2 = 2.8$
Verification:

$$3(1.8) + 2(2.8) = 5.4 + 5.6 = 11 ✓$$
$$1(1.8) + 4(2.8) = 1.8 + 11.2 = 13 ✓$$

1.5 Machine Learning Application: Linear Regression

ML Application: Ordinary Least Squares

Linear regression finds the best-fit line by solving a system of linear equations.

Example 1.4 (Linear Regression - Complete Derivation and Solution)**.** Given data points: $(x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)$

Find parameters θ_0 (intercept) and θ_1 (slope) for:

$$\hat{y} = \theta_0 + \theta_1 x$$

Problem Formulation:
Minimize sum of squared errors:

$$J(\theta_0, \theta_1) = \sum_{i=1}^{m} (y_i - \theta_0 - \theta_1 x_i)^2$$

Step 1: Derive Normal Equations
Take partial derivatives and set to zero:

$$\frac{\partial J}{\partial \theta_0} = -2\sum_{i=1}^{m}(y_i - \theta_0 - \theta_1 x_i) = 0$$

$$\frac{\partial J}{\partial \theta_1} = -2\sum_{i=1}^{m} x_i(y_i - \theta_0 - \theta_1 x_i) = 0$$

Simplifying:

$$m\theta_0 + \theta_1 \sum_{i=1}^{m} x_i = \sum_{i=1}^{m} y_i \tag{1}$$

$$\theta_0 \sum_{i=1}^{m} x_i + \theta_1 \sum_{i=1}^{m} x_i^2 = \sum_{i=1}^{m} x_i y_i \tag{2}$$

This is a system of 2 linear equations in 2 unknowns!
Matrix Form:

$$\begin{bmatrix} m & \sum x_i \\ \sum x_i & \sum x_i^2 \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum x_i y_i \end{bmatrix}$$

Step 2: Numerical Example
Dataset:

x	y
1	2
2	4
3	5
4	4
5	5

Calculate summations:

$$m = 5$$
$$\sum x_i = 1 + 2 + 3 + 4 + 5 = 15$$
$$\sum y_i = 2 + 4 + 5 + 4 + 5 = 20$$
$$\sum x_i^2 = 1 + 4 + 9 + 16 + 25 = 55$$
$$\sum x_i y_i = 2 + 8 + 15 + 16 + 25 = 66$$

System becomes:

$$\begin{bmatrix} 5 & 15 \\ 15 & 55 \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} = \begin{bmatrix} 20 \\ 66 \end{bmatrix}$$

Step 3: Solve Using Gaussian Elimination
Augmented matrix:

$$\left[\begin{array}{cc|c} 5 & 15 & 20 \\ 15 & 55 & 66 \end{array}\right]$$

$R_2 \leftarrow R_2 - 3R_1$:

$$R_2 = [15, 55, 66] - 3[5, 15, 20] = [0, 10, 6]$$

$$\left[\begin{array}{cc|c} 5 & 15 & 20 \\ 0 & 10 & 6 \end{array}\right]$$

Back substitution:

$$10\theta_1 = 6 \Rightarrow \theta_1 = 0.6$$
$$5\theta_0 + 15(0.6) = 20 \Rightarrow \theta_0 = \frac{20-9}{5} = 2.2$$

Solution: $\hat{y} = 2.2 + 0.6x$
Step 4: Predictions
For each data point:

$$x = 1 : \hat{y} = 2.2 + 0.6(1) = 2.8$$
$$x = 2 : \hat{y} = 2.2 + 0.6(2) = 3.4$$
$$x = 3 : \hat{y} = 2.2 + 0.6(3) = 4.0$$
$$x = 4 : \hat{y} = 2.2 + 0.6(4) = 4.6$$
$$x = 5 : \hat{y} = 2.2 + 0.6(5) = 5.2$$

Step 5: Calculate Error
Mean Squared Error:

$$MSE = \frac{1}{5}\sum_{i=1}^{5}(y_i - \hat{y}_i)^2 = \frac{1}{5}[(2-2.8)^2 + (4-3.4)^2 + (5-4.0)^2 + (4-4.6)^2 + (5-5.2)^2]$$

$$= \frac{1}{5}[0.64 + 0.36 + 1.00 + 0.36 + 0.04] = \frac{2.4}{5} = 0.48$$

1.6 Deep Learning Application: Finding Optimal Weights

Example 1.5 (Neural Network Weight Update via Linear System)**.** Consider a simple neural network with one hidden layer. During backpropagation, we need to find weight updates.

Problem Setup:

- Input: $\mathbf{x} = [1, 2]^T$
- Target output: $\mathbf{y} = [3, 4]^T$
- Current weights: $\mathbf{W}$
- Learning rate: $\alpha = 0.1$

Forward Pass:

$$\mathbf{z} = \mathbf{W}\mathbf{x}, \quad \hat{\mathbf{y}} = \mathbf{z}$$

Loss:

$$L = \frac{1}{2}\|\mathbf{y} - \hat{\mathbf{y}}\|^2$$

Gradient:

$$\frac{\partial L}{\partial \mathbf{W}} = -(\mathbf{y} - \hat{\mathbf{y}})\mathbf{x}^T$$

Weight Update (Linear System):

$$\mathbf{W}^{new} = \mathbf{W}^{old} + \alpha \cdot (\mathbf{y} - \hat{\mathbf{y}})\mathbf{x}^T$$

This can be formulated as solving:

$$\mathbf{W}^{new}\mathbf{x} \approx \mathbf{y}$$

Numerical Computation:

Suppose $\mathbf{W}^{old} = \begin{bmatrix} 0.5 & 0.3 \\ 0.7 & 0.9 \end{bmatrix}$

Forward pass:

$$\hat{\mathbf{y}} = \begin{bmatrix} 0.5 & 0.3 \\ 0.7 & 0.9 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1.1 \\ 2.5 \end{bmatrix}$$

Error:

$$\mathbf{y} - \hat{\mathbf{y}} = \begin{bmatrix} 3 \\ 4 \end{bmatrix} - \begin{bmatrix} 1.1 \\ 2.5 \end{bmatrix} = \begin{bmatrix} 1.9 \\ 1.5 \end{bmatrix}$$

Gradient:

$$\frac{\partial L}{\partial \mathbf{W}} = -\begin{bmatrix} 1.9 \\ 1.5 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} = -\begin{bmatrix} 1.9 & 3.8 \\ 1.5 & 3.0 \end{bmatrix}$$

Weight update:

$$\mathbf{W}^{new} = \begin{bmatrix} 0.5 & 0.3 \\ 0.7 & 0.9 \end{bmatrix} + 0.1 \begin{bmatrix} 1.9 & 3.8 \\ 1.5 & 3.0 \end{bmatrix} = \begin{bmatrix} 0.69 & 0.68 \\ 0.85 & 1.20 \end{bmatrix}$$

1.7 Computational Complexity Analysis

Method	**Time Complexity**	**Space Complexity**
Gaussian Elimination	$O(n^3)$	$O(n^2)$
LU Decomposition	$O(n^3)$	$O(n^2)$
Matrix Inverse	$O(n^3)$	$O(n^2)$
Back Substitution	$O(n^2)$	$O(n)$
Forward Substitution	$O(n^2)$	$O(n)$

2 Linear Transformations

2.1 Introduction and Motivation

Why Linear Transformations in ML/DL?

Linear transformations map input data to different spaces while preserving relationships between points. This is fundamental in machine learning and deep learning algorithms, from simple regression to complex neural networks. Each layer in a neural network performs a linear transformation followed by a non-linear activation.

2.2 Fundamental Definitions

Definition 2.1 (Linear Transformation)**.** A function $T : \mathbb{R}^n \to \mathbb{R}^m$ is called a linear transformation if for all vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ and all scalars $c \in \mathbb{R}$:

1. $T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v})$ (Additivity)
2. $T(c\mathbf{u}) = cT(\mathbf{u})$ (Homogeneity)

Equivalently: $T(c_1\mathbf{u} + c_2\mathbf{v}) = c_1T(\mathbf{u}) + c_2T(\mathbf{v})$ (Linearity)

Theorem 2.1 (Matrix Representation)**.** Every linear transformation $T : \mathbb{R}^n \to \mathbb{R}^m$ can be represented by a unique matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ such that:

$$T(\mathbf{x}) = \mathbf{A}\mathbf{x}$$

The columns of $\mathbf{A}$ are $T(\mathbf{e}_1), T(\mathbf{e}_2), \ldots, T(\mathbf{e}_n)$ where $\mathbf{e}_i$ are standard basis vectors.

2.3 Properties of Linear Transformations

Property 2.1 (Important Properties)**.**

1. $T(\mathbf{0}) = \mathbf{0}$ (Zero vector maps to zero vector)
2. $T(-\mathbf{v}) = -T(\mathbf{v})$ (Preserves negation)
3. $T(\mathbf{v} - \mathbf{w}) = T(\mathbf{v}) - T(\mathbf{w})$ (Preserves differences)
4. Lines map to lines (or points)
5. Parallel lines map to parallel lines (or points)
6. Origin is always fixed

2.4 Types of Linear Transformations in 2D and 3D

2.4.1 Scaling

Definition 2.2 (Scaling Transformation)**.** Stretches or compresses along coordinate axes:

$$\mathbf{A}_{scale} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

where s_x, s_y are scaling factors.

Example 2.1 (Scaling in 2D)**.** Scale a point by factor 2 in x-direction and 3 in y-direction:

$$\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$$

Transform point $\mathbf{v} = [1, 2]^T$:

$$T(\mathbf{v}) = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 2 \\ 6 \end{bmatrix}$$

ML Application - Feature Scaling:
In machine learning, feature scaling standardizes features:

$$\mathbf{x}_{scaled} = \text{diag}(\mathbf{s}) \cdot \mathbf{x}$$

Example dataset with 3 features:

Age	Income	Score
25	50000	85
35	75000	92
45	100000	78

Scaling factors to normalize to [0,1]:

$$s_1 = \frac{1}{45 - 25} = 0.05$$
$$s_2 = \frac{1}{100000 - 50000} = 0.00002$$
$$s_3 = \frac{1}{92 - 78} = 0.071$$

For point [35, 75000, 92]:

$$\begin{bmatrix} 0.05 & 0 & 0 \\ 0 & 0.00002 & 0 \\ 0 & 0 & 0.071 \end{bmatrix} \begin{bmatrix} 35 - 25 \\ 75000 - 50000 \\ 92 - 78 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.99 \end{bmatrix}$$

2.4.2 Rotation

Definition 2.3 (Rotation Transformation in 2D)**.** Rotates vectors by angle θ counterclockwise:

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

Example 2.2 (Rotation by 90 degrees)**.** Rotate point $[3, 1]^T$ by 90° counterclockwise:
Step 1: $\theta = 90 = \frac{\pi}{2}$ radians
Step 2: Construct rotation matrix:

$$\mathbf{R}(90) = \begin{bmatrix} \cos(90) & -\sin(90) \\ \sin(90) & \cos(90) \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

Step 3: Apply transformation:

$$\mathbf{v}' = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 0(3) + (-1)(1) \\ 1(3) + 0(1) \end{bmatrix} = \begin{bmatrix} -1 \\ 3 \end{bmatrix}$$

Verification: Original point $(3, 1)$ is at distance $\sqrt{10}$ from origin. After rotation: $\sqrt{(-1)^2 + 3^2} = \sqrt{10}$

ML Application - Data Augmentation:
In computer vision, rotating images for data augmentation:

$$\begin{bmatrix} \cos(30) & -\sin(30) \\ \sin(30) & \cos(30) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}_{\text{pixel}} = \begin{bmatrix} 0.866x - 0.5y \\ 0.5x + 0.866y \end{bmatrix}$$

2.4.3 Reflection

Definition 2.4 (Reflection Transformations)**.** Reflection across x-axis: $\mathbf{R}_x = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$

Reflection across y-axis: $\mathbf{R}_y = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix}$

Reflection across $y = x$: $\mathbf{R}_{y=x} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$

2.4.4 Shear

Definition 2.5 (Shear Transformation)**.** Horizontal shear: $\mathbf{S}_x = \begin{bmatrix} 1 & k \\ 0 & 1 \end{bmatrix}$

Vertical shear: $\mathbf{S}_y = \begin{bmatrix} 1 & 0 \\ k & 1 \end{bmatrix}$

where k is the shear factor.

Example 2.3 (Shear Transformation)**.** Apply horizontal shear with $k = 2$ to point $[1, 2]^T$:

$$\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 + 2(2) \\ 2 \end{bmatrix} = \begin{bmatrix} 5 \\ 2 \end{bmatrix}$$

The x-coordinate shifts by $2 \times y = 4$ units.

2.5 Projection

Definition 2.6 (Orthogonal Projection)**.** Projection onto a line in direction $\mathbf{u}$ (unit vector):

$$\text{proj}_{\mathbf{u}}(\mathbf{v}) = (\mathbf{v} \cdot \mathbf{u})\mathbf{u} = (\mathbf{u}\mathbf{u}^T)\mathbf{v}$$

Matrix form: $\mathbf{P} = \mathbf{u}\mathbf{u}^T$

Example 2.4 (Projection onto a Line)**.** Project $\mathbf{v} = [3, 4]^T$ onto the line $y = x$ (i.e., direction $\mathbf{u} = \frac{1}{\sqrt{2}}[1, 1]^T$):

Step 1: Verify $\mathbf{u}$ is unit vector:

$$\|\mathbf{u}\| = \sqrt{\left(\frac{1}{\sqrt{2}}\right)^2 + \left(\frac{1}{\sqrt{2}}\right)^2} = \sqrt{\frac{1}{2} + \frac{1}{2}} = 1 \checkmark$$

Step 2: Construct projection matrix:

$$\mathbf{P} = \mathbf{u}\mathbf{u}^T = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

Step 3: Apply projection:

$$\mathbf{Pv} = \frac{1}{2}\begin{bmatrix}1 & 1\\ 1 & 1\end{bmatrix}\begin{bmatrix}3\\ 4\end{bmatrix} = \frac{1}{2}\begin{bmatrix}7\\ 7\end{bmatrix} = \begin{bmatrix}3.5\\ 3.5\end{bmatrix}$$

Interpretation: Point $(3, 4)$ projects to $(3.5, 3.5)$ on line $y = x$.
ML Application - PCA:
Principal Component Analysis uses projections:

$$\mathbf{x}_{projected} = \mathbf{P}_k\mathbf{x} = \mathbf{U}_k\mathbf{U}_k^T\mathbf{x}$$

where $\mathbf{U}_k$ contains top k principal components.

2.6 Kernel and Image

Definition 2.7 (Kernel (Null Space))**.** The kernel of a linear transformation T is:

$$\ker(T) = \{\mathbf{v} \in \mathbb{R}^n : T(\mathbf{v}) = \mathbf{0}\}$$

All vectors that map to the zero vector.

Definition 2.8 (Image (Range))**.** The image of T is:

$$\text{Im}(T) = \{T(\mathbf{v}) : \mathbf{v} \in \mathbb{R}^n\}$$

All possible outputs of the transformation.

Example 2.5 (Finding Kernel and Image)**.** Consider transformation $T : \mathbb{R}^3 \to \mathbb{R}^2$ with matrix:

$$\mathbf{A} = \begin{bmatrix}1 & 2 & 3\\ 2 & 4 & 6\end{bmatrix}$$

Find Kernel:
Solve $\mathbf{Ax} = \mathbf{0}$:

$$\begin{bmatrix}1 & 2 & 3\\ 2 & 4 & 6\end{bmatrix}\begin{bmatrix}x_1\\ x_2\\ x_3\end{bmatrix} = \begin{bmatrix}0\\ 0\end{bmatrix}$$

Row reduce:

$$\left[\begin{array}{ccc|c}1 & 2 & 3 & 0\\ 2 & 4 & 6 & 0\end{array}\right] \to \left[\begin{array}{ccc|c}1 & 2 & 3 & 0\\ 0 & 0 & 0 & 0\end{array}\right]$$

From $x_1 + 2x_2 + 3x_3 = 0$, we get $x_1 = -2x_2 - 3x_3$
Kernel:

$$\ker(T) = \text{span}\left\{\begin{bmatrix}-2\\ 1\\ 0\end{bmatrix}, \begin{bmatrix}-3\\ 0\\ 1\end{bmatrix}\right\}$$

Find Image:
Image is spanned by linearly independent columns. Since column 2 = 2×column 1 and column 3 = 3×column 1:

$$\text{Im}(T) = \text{span}\left\{\begin{bmatrix}1\\ 2\end{bmatrix}\right\}$$

Dimension: $\dim(\ker(T)) = 2$, $\dim(\text{Im}(T)) = 1$
Verify rank-nullity: $2 + 1 = 3 = n$

2.7 Machine Learning Applications

Example 2.6 (Feature Transformation in ML)**. Problem**: Transform 2D data for better linear separability.

Original features: $\mathbf{x} = [x_1, x_2]^T$

Dataset:

x_1	x_2	Class
1	1	A
2	1	A
1	2	B
2	2	B

Apply Linear Transformation:

$$\mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

This creates features: $z_1 = x_1 + x_2$ and $z_2 = x_1 - x_2$

Transformed Dataset:

$$\mathbf{z}_1 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix} \quad \text{(Class A)}$$

$$\mathbf{z}_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix} \quad \text{(Class A)}$$

$$\mathbf{z}_3 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 3 \\ -1 \end{bmatrix} \quad \text{(Class B)}$$

$$\mathbf{z}_4 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \end{bmatrix} \quad \text{(Class B)}$$

Now classes are more separable in the z_2 dimension!

Example 2.7 (Neural Network Layer as Linear Transformation)**. Single Layer Neural Network**:

Input: $\mathbf{x} = [0.5, 0.8, 0.3]^T$

Weight matrix:

$$\mathbf{W} = \begin{bmatrix} 0.2 & 0.5 & -0.3 \\ 0.7 & -0.1 & 0.4 \end{bmatrix}$$

Bias: $\mathbf{b} = [0.1, -0.2]^T$

Linear Transformation (pre-activation):

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

Calculation:

$$\mathbf{z} = \begin{bmatrix} 0.2 & 0.5 & -0.3 \\ 0.7 & -0.1 & 0.4 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.8 \\ 0.3 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

$$= \begin{bmatrix} 0.2(0.5) + 0.5(0.8) + (-0.3)(0.3) \\ 0.7(0.5) + (-0.1)(0.8) + 0.4(0.3) \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$$

$$= \begin{bmatrix} 0.1 + 0.4 - 0.09 \\ 0.35 - 0.08 + 0.12 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix} = \begin{bmatrix} 0.41 \\ 0.39 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix} = \begin{bmatrix} 0.51 \\ 0.19 \end{bmatrix}$$

After Activation (ReLU):

$$\mathbf{a} = \text{ReLU}(\mathbf{z}) = \begin{bmatrix} \max(0, 0.51) \\ \max(0, 0.19) \end{bmatrix} = \begin{bmatrix} 0.51 \\ 0.19 \end{bmatrix}$$

The linear transformation $\mathbf{W}\mathbf{x} + \mathbf{b}$ maps 3D input to 2D hidden representation.

3 3D Linear Transformations

3.1 Introduction

3D Transformations in ML/DL

3D transformations preserve points, lines, and planes. They're essential in computer vision for object recognition, 3D reconstruction, visualization, and geometric interpretation of data in 3D space.

3.2 Basic 3D Transformations

3.2.1 3D Rotation Matrices

Definition 3.1 (Rotation about Coordinate Axes)**.** Rotation about x-axis by angle θ:

$$\mathbf{R}_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix}$$

Rotation about y-axis by angle θ:

$$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

Rotation about z-axis by angle θ:

$$\mathbf{R}_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Example 3.1 (3D Rotation - Complete Calculation)**.** Rotate point $\mathbf{p} = [1, 0, 0]^T$ by $90°$ about the z-axis:

Step 1: Identify rotation matrix ($\theta = 90 = \pi/2$):

$$\mathbf{R}_z(90) = \begin{bmatrix} \cos(90) & -\sin(90) & 0 \\ \sin(90) & \cos(90) & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Step 2: Apply transformation:

$$\mathbf{p}' = \mathbf{R}_z(90)\mathbf{p} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Point $(1, 0, 0)$ rotates to $(0, 1, 0)$ on the xy-plane.

Geometric Interpretation: The point moves from positive x-axis to positive y-axis while maintaining z-coordinate.

Example 3.2 (Multiple 3D Rotations)**.** Rotate point $[1,1,1]^T$ first by 90° about x-axis, then 45° about z-axis:

Step 1: First rotation (x-axis, 90°):

$$\mathbf{R}_x(90) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$$

$$\mathbf{p}_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix}$$

Step 2: Second rotation (z-axis, 45°):

$$\mathbf{R}_z(45) = \begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{p}_2 = \begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{2}} + \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} - \frac{1}{\sqrt{2}} \\ 1 \end{bmatrix} = \begin{bmatrix} \sqrt{2} \\ 0 \\ 1 \end{bmatrix}$$

Combined Rotation:

$$\mathbf{R}_{combined} = \mathbf{R}_z(45)\mathbf{R}_x(90)$$

3.2.2 3D Scaling

Definition 3.2 (3D Scaling Matrix)**.**

$$\mathbf{S}(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \end{bmatrix}$$

Example 3.3 (3D Scaling for 3D Point Cloud)**.** Scale a 3D point cloud representing an object:

Points: $\mathbf{p}_1 = [1,2,3]^T$, $\mathbf{p}_2 = [2,3,4]^T$

Scaling: $s_x = 2, s_y = 2, s_z = 0.5$ (compress in z, expand in x,y)

$$\mathbf{S} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0.5 \end{bmatrix}$$

$$\mathbf{p}_1' = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0.5 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \\ 1.5 \end{bmatrix}$$

$$\mathbf{p}_2' = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0.5 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 4 \\ 6 \\ 2 \end{bmatrix}$$

ML Application: Used in 3D object augmentation for training robust 3D detectors.

3.3 Computer Vision Application

Example 3.4 (Camera Projection - 3D to 2D)**. Problem**: Project 3D world point to 2D image plane.

3D point in world: $\mathbf{P}_w = [X, Y, Z]^T$

Camera at origin, looking down z-axis, focal length $f = 500$ pixels.

Perspective Projection:

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \frac{1}{Z} \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Numerical Example:

Point: $\mathbf{P}_w = [100, 200, 1000]^T$ (in mm)

$$\begin{bmatrix} u \\ v \\ w \end{bmatrix} = \begin{bmatrix} 500 & 0 & 0 & 0 \\ 0 & 500 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 100 \\ 200 \\ 1000 \\ 1 \end{bmatrix} = \begin{bmatrix} 50000 \\ 100000 \\ 1000 \end{bmatrix}$$

Convert to pixel coordinates:

$$u = \frac{50000}{1000} = 50, \quad v = \frac{100000}{1000} = 100$$

Image point: $(50, 100)$ pixels

4 Matrix Multiplication as Composition

4.1 Introduction and Significance

Composition in ML/DL

In machine learning and deep learning, sequential transformations are represented by matrix multiplication. Each layer of a neural network performs a transformation, and the composition represents the entire network. This is fundamental to understanding deep architectures.

4.2 Composition of Linear Transformations

Definition 4.1 (Composition)**.** If $T_1 : \mathbb{R}^n \to \mathbb{R}^m$ and $T_2 : \mathbb{R}^m \to \mathbb{R}^p$ are linear transformations with matrices $\mathbf{A}_1$ and $\mathbf{A}_2$, their composition is:

$$(T_2 \circ T_1)(\mathbf{x}) = T_2(T_1(\mathbf{x})) = \mathbf{A}_2(\mathbf{A}_1\mathbf{x}) = (\mathbf{A}_2\mathbf{A}_1)\mathbf{x}$$

The matrix of the composition is $\mathbf{A}_2\mathbf{A}_1$.

Property 4.1 (Properties of Composition)**.**

1. **Associativity**: $(\mathbf{A}_3\mathbf{A}_2)\mathbf{A}_1 = \mathbf{A}_3(\mathbf{A}_2\mathbf{A}_1)$
2. **Non-commutativity**: Generally $\mathbf{A}_2\mathbf{A}_1 \neq \mathbf{A}_1\mathbf{A}_2$
3. **Identity**: $\mathbf{IA} = \mathbf{AI} = \mathbf{A}$
4. **Inverse of composition**: $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$

Example 4.1 (Simple Composition - Geometric Transformations)**.** Apply scaling then rotation to point $[1, 0]^T$:

Transformation 1 - Scale by 2:

$$\mathbf{S} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

Transformation 2 - Rotate 90°:

$$\mathbf{R} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

Method 1: Sequential Application

Step 1 - Scale:

$$\mathbf{v}_1 = \mathbf{S}\mathbf{v} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

Step 2 - Rotate:

$$\mathbf{v}_2 = \mathbf{R}\mathbf{v}_1 = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$$

Method 2: Composed Matrix

$$\mathbf{C} = \mathbf{RS} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} 0(2) + (-1)(0) & 0(0) + (-1)(2) \\ 1(2) + 0(0) & 1(0) + 0(2) \end{bmatrix} = \begin{bmatrix} 0 & -2 \\ 2 & 0 \end{bmatrix}$$

Apply composed transformation:

$$\mathbf{v}_2 = \mathbf{C}\mathbf{v} = \begin{bmatrix} 0 & -2 \\ 2 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$$

Both methods give the same result!
Compare: Different Order
If we rotate then scale (**SR**):

$$\mathbf{C}' = \mathbf{SR} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & -2 \\ 2 & 0 \end{bmatrix}$$

In this case $\mathbf{SR} = \mathbf{RS}$ because scaling is uniform. But generally, order matters!

Example 4.2 (Non-Commutativity of Matrix Multiplication)**. Transformation 1**: Horizontal shear with $k = 1$

$$\mathbf{H} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

Transformation 2: Scale by $(2, 1)$

$$\mathbf{S} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$$

Composition 1: **SH** (shear then scale)

$$\mathbf{SH} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 2 \\ 0 & 1 \end{bmatrix}$$

Composition 2: **HS** (scale then shear)

$$\mathbf{HS} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}$$

Clearly $\mathbf{SH} \neq \mathbf{HS}$! Order matters.
Apply to point $[1, 1]^T$:
SH gives:

$$\begin{bmatrix} 2 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \end{bmatrix}$$

HS gives:

$$\begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

Different results!

4.3 Deep Learning Application: Multi-Layer Neural Networks

Example 4.3 (Two-Layer Neural Network as Matrix Composition)**. Architecture**:

- Input layer: 3 neurons
- Hidden layer: 4 neurons
- Output layer: 2 neurons

Weight Matrices:
Layer 1 (input to hidden):

$$\mathbf{W}_1 = \begin{bmatrix} 0.5 & 0.2 & -0.3 \\ -0.1 & 0.4 & 0.6 \\ 0.3 & -0.2 & 0.1 \\ 0.7 & 0.5 & -0.4 \end{bmatrix} \in \mathbb{R}^{4\times 3}$$

Layer 2 (hidden to output):

$$\mathbf{W}_2 = \begin{bmatrix} 0.8 & -0.2 & 0.5 & 0.3 \\ -0.4 & 0.6 & 0.1 & -0.3 \end{bmatrix} \in \mathbb{R}^{2\times 4}$$

Input:

$$\mathbf{x} = \begin{bmatrix} 0.5 \\ 0.3 \\ 0.8 \end{bmatrix}$$

Forward Pass - Step by Step:
Step 1: Input to hidden (linear):

$$\mathbf{z}_1 = \mathbf{W}_1\mathbf{x} = \begin{bmatrix} 0.5 & 0.2 & -0.3 \\ -0.1 & 0.4 & 0.6 \\ 0.3 & -0.2 & 0.1 \\ 0.7 & 0.5 & -0.4 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.3 \\ 0.8 \end{bmatrix}$$

Calculating each element:

$$\begin{aligned} z_{1,1} &= 0.5(0.5) + 0.2(0.3) + (-0.3)(0.8) = 0.25 + 0.06 - 0.24 = 0.07 \\ z_{1,2} &= -0.1(0.5) + 0.4(0.3) + 0.6(0.8) = -0.05 + 0.12 + 0.48 = 0.55 \\ z_{1,3} &= 0.3(0.5) + (-0.2)(0.3) + 0.1(0.8) = 0.15 - 0.06 + 0.08 = 0.17 \\ z_{1,4} &= 0.7(0.5) + 0.5(0.3) + (-0.4)(0.8) = 0.35 + 0.15 - 0.32 = 0.18 \end{aligned}$$

$$\mathbf{z}_1 = \begin{bmatrix} 0.07 \\ 0.55 \\ 0.17 \\ 0.18 \end{bmatrix}$$

Step 2: Apply activation (ReLU):

$$\mathbf{a}_1 = \text{ReLU}(\mathbf{z}_1) = \begin{bmatrix} 0.07 \\ 0.55 \\ 0.17 \\ 0.18 \end{bmatrix}$$

(all positive, so unchanged)
Step 3: Hidden to output (linear):

$$\mathbf{z}_2 = \mathbf{W}_2\mathbf{a}_1 = \begin{bmatrix} 0.8 & -0.2 & 0.5 & 0.3 \\ -0.4 & 0.6 & 0.1 & -0.3 \end{bmatrix} \begin{bmatrix} 0.07 \\ 0.55 \\ 0.17 \\ 0.18 \end{bmatrix}$$

Calculating:

$$\begin{aligned} z_{2,1} &= 0.8(0.07) + (-0.2)(0.55) + 0.5(0.17) + 0.3(0.18) \\ &= 0.056 - 0.11 + 0.085 + 0.054 = 0.085 \\ z_{2,2} &= -0.4(0.07) + 0.6(0.55) + 0.1(0.17) + (-0.3)(0.18) \\ &= -0.028 + 0.33 + 0.017 - 0.054 = 0.265 \end{aligned}$$

$$\mathbf{z}_2 = \begin{bmatrix} 0.085 \\ 0.265 \end{bmatrix}$$

Interpretation of Composition:
Without activation functions, the network would be:

$$\mathbf{z}_2 = \mathbf{W}_2(\mathbf{W}_1\mathbf{x}) = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x}$$

This is just a single linear transformation! This is why activation functions are crucial—they break the linearity.

Effective Weight Matrix (if no activation):

$$\mathbf{W}_{eff} = \mathbf{W}_2\mathbf{W}_1 \in \mathbb{R}^{2\times 3}$$

This demonstrates why deep networks need non-linearities!

Example 4.4 (Convolutional Layer as Matrix Multiplication)**. Problem**: Understand convolution as matrix operation.

Input: 3×3 image

$$\mathbf{I} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel: 2×2 filter

$$\mathbf{K} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Flatten image to vector:

$$\mathbf{x} = [1, 2, 3, 4, 5, 6, 7, 8, 9]^T$$

Construct Toeplitz matrix for convolution:
Each row corresponds to one position of the filter:

$$\mathbf{C} = \begin{bmatrix} 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & -1 \end{bmatrix}$$

Apply convolution as matrix multiplication:

$$\mathbf{y} = \mathbf{C}\mathbf{x} = \begin{bmatrix} 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \end{bmatrix}$$

Computing each output:

$$\begin{aligned} y_1 &= 1(1) + (-1)(5) = 1 - 5 = -4 \\ y_2 &= 1(2) + (-1)(6) = 2 - 6 = -4 \\ y_3 &= 1(4) + (-1)(8) = 4 - 8 = -4 \\ y_4 &= 1(5) + (-1)(9) = 5 - 9 = -4 \end{aligned}$$

$$\mathbf{y} = \begin{bmatrix} -4 \\ -4 \\ -4 \\ -4 \end{bmatrix}$$

Reshape to 2×2:

$$\mathbf{Y} = \begin{bmatrix} -4 & -4 \\ -4 & -4 \end{bmatrix}$$

This shows convolution is a linear transformation!

4.4 Practical Implications

Key Takeaways for ML/DL

1. **Network Depth**: Multiple layers compose transformations, creating complex decision boundaries
2. **Parameter Efficiency**: Composition allows learning hierarchical features
3. **Gradient Flow**: Backpropagation uses chain rule through composed transformations
4. **Geometric Interpretation**: Each layer transforms data geometry
5. **Non-commutativity**: Layer order matters significantly

4.5 Computational Efficiency

Example 4.5 (Efficiency of Matrix Composition)**. Scenario**: Apply 3 transformations to 1000 data points.

Matrices:

- $\mathbf{A}_1 \in \mathbb{R}^{100 \times 50}$
- $\mathbf{A}_2 \in \mathbb{R}^{75 \times 100}$
- $\mathbf{A}_3 \in \mathbb{R}^{50 \times 75}$

Data: $\mathbf{X} \in \mathbb{R}^{50 \times 1000}$ (50 features, 1000 samples)
Method 1 - Sequential: Apply to each point separately

$$\mathbf{Y} = \mathbf{A}_3(\mathbf{A}_2(\mathbf{A}_1\mathbf{X}))$$

Operations:

$$\mathbf{A}_1\mathbf{X} : 100 \times 50 \times 1000 = 5,000,000 \text{ ops}$$
$$\mathbf{A}_2(\cdot) : 75 \times 100 \times 1000 = 7,500,000 \text{ ops}$$
$$\mathbf{A}_3(\cdot) : 50 \times 75 \times 1000 = 3,750,000 \text{ ops}$$

Total: $16,250,000$ operations
Method 2 - Pre-compose (if reusing):

$$\mathbf{A}_{comp} = \mathbf{A}_3\mathbf{A}_2\mathbf{A}_1$$

Composition cost:

$$\mathbf{A}_2\mathbf{A}_1 : 75 \times 100 \times 50 = 375,000 \text{ ops}$$
$$\mathbf{A}_3(\cdot) : 50 \times 75 \times 100 = 375,000 \text{ ops}$$

Composition total: $750,000$ ops
Apply to data: $50 \times 50 \times 1000 = 2,500,000$ ops
Total: $3,250,000$ operations ($5\times$ faster!)
Lesson: Pre-composition is efficient when transforming many points with same sequence.

5 Summary and Connections

Unified View

Solving Linear Systems $\leftrightarrow$ Finding optimal parameters (regression, neural networks)
Linear Transformations $\leftrightarrow$ Feature mapping, data preprocessing, layer operations
3D Transformations $\leftrightarrow$ Computer vision, 3D understanding, geometric deep learning
Matrix Composition $\leftrightarrow$ Multi-layer networks, transformation pipelines, transfer learning

5.1 Practical Checklist for ML/DL Practitioners

1. When solving for model parameters, consider:
   - System solvability (existence/uniqueness)
   - Numerical stability
   - Computational complexity
2. When designing transformations:
   - Preserve important properties
   - Consider invertibility
   - Check for information loss

3. When composing layers:
    - Order matters (non-commutativity)
    - Add non-linearities between linear layers
    - Monitor gradient flow
4. When working with 3D data:
    - Use appropriate augmentations (rotation, scaling)
    - Preserve geometric relationships
    - Consider computational cost

6 Assessment Questions

6.1 Level 1: Basic Understanding

1. **What is a system of linear equations?** Provide two examples from machine learning.
2. **Give three methods** for solving a system of linear equations and their computational complexities.
3. **What is the difference between a linear transformation and a general function?** Provide examples.
4. **Calculate**: Solve the system:
$$\begin{cases} 2x + y = 5 \\ x - y = 1 \end{cases}$$
5. **If** $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$, find $\mathbf{Av}$.
6. **True or False**: Every linear transformation can be represented by a matrix.
7. **Identify whether each is a linear transformation**:
   (a) $T(x) = 2x + 3$
   (b) $T(x, y) = (2x, 3y)$
   (c) $T(x) = x^2$
8. **Compute**: $\mathbf{R}(90) \begin{bmatrix} 1 \\ 0 \end{bmatrix}$ where $\mathbf{R}$ is rotation matrix.
9. **What are the two fundamental properties** that define a linear transformation?
10. **Why is matrix multiplication** generally non-commutative? Give an example.

6.2 Level 2: Intermediate

11. **What is the role of solving linear systems in linear regression?** Provide the normal equations.
12. **Compute** the LU decomposition of $\mathbf{A} = \begin{bmatrix} 2 & 4 \\ 1 & 3 \end{bmatrix}$.
13. **State the projection formula** and explain its meaning in PCA.
14. **Given** $\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$ and $\mathbf{B} = \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix}$, compute $\mathbf{AB}$ and $\mathbf{BA}$.
15. **What is the kernel (null space)** of a linear transformation? Give geometric interpretation.
16. **Given transformation** $T(x, y) = (x + 2y, 3x - y)$, find its matrix representation.

17. **Calculate**: Apply shear transformation with $k = 2$ to point $(1, 3)$.

18. **If** $\mathbf{W} = \begin{bmatrix} 0.5 & -0.3 \\ 0.2 & 0.8 \end{bmatrix}$, find $\mathbf{Wx}$ where $\mathbf{x} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

19. **Solve using Gaussian elimination**:

$$\begin{cases} x + 2y - z = 3 \\ 2x + y + z = 1 \\ x - y + z = -1 \end{cases}$$

20. **Calculate** the composition of scaling by 2 and rotation by 45°.

6.3 Level 3: Advanced

21. **Prove** that the composition of linear transformations is associative: $(T_3 \circ T_2) \circ T_1 = T_3 \circ (T_2 \circ T_1)$.

22. **Why does a neural network without activation functions** collapse to a single linear transformation?

23. **Explain the role of matrix multiplication** in the backpropagation algorithm for multi-layer networks.

24. **Compare the computational complexity** of solving $\mathbf{Ax} = \mathbf{b}$ using:
    (a) Gaussian elimination
    (b) Matrix inverse
    (c) LU decomposition

25. **Design a feature transformation strategy** using linear transformations to make data linearly separable.

26. **Explain how 3D transformations** are used in data augmentation for computer vision. Provide specific transformation matrices.

27. **What happens to the determinant** when you compose two transformations? Prove your answer.

28. **Prove** that matrix multiplication is distributive: $\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$.

29. **Explain the relationship between eigenvalues** and the behavior of iterative methods for solving linear systems.

30. **How do you determine** if a system of linear equations is over-determined, under-determined, or exactly determined? What are the implications for machine learning?

6.4 Multiple Choice Questions

31. The solution to $\mathbf{Ax} = \mathbf{b}$ is unique if and only if:

    (a) $\mathbf{A}$ is square

    (b) $\mathbf{A}$ is invertible

    (c) $\mathbf{A}$ is symmetric

    (d) $\mathbf{A}$ is diagonal

32. In neural networks, if all layers use only linear transformations without activations:

    (a) The network can learn any function

    (b) The network reduces to a single linear transformation

    (c) The network cannot be trained

    (d) The network trains faster

33. The rank-nullity theorem states:

    (a) $\text{rank}(\mathbf{A}) + \text{nullity}(\mathbf{A}) = \text{number of rows}$

    (b) $\text{rank}(\mathbf{A}) + \text{nullity}(\mathbf{A}) = \text{number of columns}$

    (c) $\text{rank}(\mathbf{A}) \times \text{nullity}(\mathbf{A}) = \text{determinant}$

    (d) $\text{rank}(\mathbf{A}) = \text{nullity}(\mathbf{A})$

34. Which of the following is NOT preserved by linear transformations?

    (a) Collinearity

    (b) Parallelism

    (c) Angles

    (d) Ratios of distances along a line

35. The computational complexity of matrix multiplication for $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$ is:

    (a) $O(mnp)$

    (b) $O(m + n + p)$

    (c) $O(\max(m, n, p))$

    (d) $O(n^2)$

6.5 Programming Exercises

P1. Implement Gaussian elimination in Python without using NumPy's built-in solvers.

P2. Write a function to compute LU decomposition and solve a system using forward and backward substitution.

P3. Create a class that represents a 2D linear transformation with methods for composition, inversion, and application to points.

P4. Implement a simple neural network layer that performs $\mathbf{y} = \mathbf{W}\mathbf{x}+\mathbf{b}$ and its gradient computation.

P5. Build a linear regression solver using the normal equations and compare with gradient descent.

7 Solutions to Assessment Questions

7.1 Level 1 Solutions

A1. A system of linear equations is a collection of two or more linear equations involving the same variables.

ML Examples:

1. *Linear Regression*: The normal equations $\mathbf{X}^T\mathbf{X}\boldsymbol{\theta} = \mathbf{X}^T\mathbf{y}$ form a system to find optimal parameters.
2. *Neural Network Weight Update*: Solving for weight updates in least-squares problems.

Explanation: Systems of linear equations are fundamental in finding optimal parameters in many ML algorithms.

A2. Three methods:

1. **Gaussian Elimination**: $O(n^3)$ time complexity
2. **LU Decomposition**: $O(n^3)$ for decomposition, $O(n^2)$ for each solve
3. **Matrix Inverse**: $O(n^3)$ time complexity

A3. A linear transformation satisfies $T(a\mathbf{u} + b\mathbf{v}) = aT(\mathbf{u}) + bT(\mathbf{v})$ for all scalars a, b and vectors $\mathbf{u}, \mathbf{v}$.

Examples:

- *Linear*: $T(x, y) = (2x + y, x - 3y)$ — satisfies additivity and homogeneity
- *Not Linear*: $T(x, y) = (x + 1, y)$ — doesn't map zero to zero
- *Not Linear*: $T(x) = x^2$ — doesn't satisfy $T(2x) = 2T(x)$

A4.

$$\begin{aligned} 2x + y &= 5 \quad ...(1) \\ x - y &= 1 \quad ...(2) \end{aligned}$$

Add equations (1) and (2):

$$(2x + y) + (x - y) = 5 + 1$$

$$3x = 6 \Rightarrow x = 2$$

Substitute into (2):

$$2 - y = 1 \Rightarrow y = 1$$

Solution: $x = 2, y = 1$

Verification:

$$\begin{aligned} 2(2) + 1 &= 5\checkmark \\ 2 - 1 &= 1\checkmark \end{aligned}$$

A5.

$$\mathbf{Av} = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 2(1) + 1(2) \\ 0(1) + 3(2) \end{bmatrix} = \begin{bmatrix} 4 \\ 6 \end{bmatrix}$$

A6. True. Every linear transformation from $\mathbb{R}^n$ to $\mathbb{R}^m$ can be uniquely represented by an $m \times n$ matrix.

Explanation: The matrix columns are the images of the standard basis vectors.

A7.

(a) $T(x) = 2x + 3$ — **NOT linear** (doesn't map 0 to 0)

(b) $T(x, y) = (2x, 3y)$ — **Linear** (satisfies both properties)

(c) $T(x) = x^2$ — **NOT linear** ($T(2x) = 4x^2 \neq 2x^2 = 2T(x)$)

A8.

$$\mathbf{R}(90) = \begin{bmatrix} \cos 90 & -\sin 90 \\ \sin 90 & \cos 90 \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

$$\mathbf{R}(90) \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Explanation: Point $(1, 0)$ on x-axis rotates to $(0, 1)$ on y-axis.

A9. The two fundamental properties are:

1. **Additivity**: $T(\mathbf{u} + \mathbf{v}) = T(\mathbf{u}) + T(\mathbf{v})$
2. **Homogeneity**: $T(c\mathbf{u}) = cT(\mathbf{u})$ for any scalar c

A10. Matrix multiplication is non-commutative because the dimensions or the transformation order matters.

Example:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 & 0 \\ 3 & 1 \end{bmatrix}$$

$$\mathbf{AB} = \begin{bmatrix} 7 & 2 \\ 3 & 1 \end{bmatrix}, \quad \mathbf{BA} = \begin{bmatrix} 1 & 2 \\ 3 & 7 \end{bmatrix}$$

Clearly $\mathbf{AB} \neq \mathbf{BA}$.

7.2 Level 2 Solutions

A11. In linear regression, we minimize $\|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2$ which leads to the normal equations:

$$\mathbf{X}^T\mathbf{X}\boldsymbol{\theta} = \mathbf{X}^T\mathbf{y}$$

This is a system of linear equations where:

- $\mathbf{X}^T\mathbf{X}$ is the coefficient matrix
- $\boldsymbol{\theta}$ is the unknown parameter vector
- $\mathbf{X}^T\mathbf{y}$ is the constant vector

Explanation: Solving this system gives the optimal least-squares solution.

A12. LU decomposition of $\mathbf{A} = \begin{bmatrix} 2 & 4 \\ 1 & 3 \end{bmatrix}$:

Step 1: First elimination step
Multiplier: $l_{21} = \frac{1}{2}$
After elimination:

$$\mathbf{U} = \begin{bmatrix} 2 & 4 \\ 0 & 1 \end{bmatrix}$$

Step 2: Record multipliers in $\mathbf{L}$:

$$\mathbf{L} = \begin{bmatrix} 1 & 0 \\ 1/2 & 1 \end{bmatrix}$$

Verification:

$$\mathbf{LU} = \begin{bmatrix} 1 & 0 \\ 1/2 & 1 \end{bmatrix} \begin{bmatrix} 2 & 4 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 1 & 3 \end{bmatrix} = \mathbf{A}\checkmark$$

A13. Projection formula: For projecting vector $\mathbf{v}$ onto unit vector $\mathbf{u}$:

$$\text{proj}_{\mathbf{u}}(\mathbf{v}) = (\mathbf{v} \cdot \mathbf{u})\mathbf{u} = (\mathbf{u}\mathbf{u}^T)\mathbf{v}$$

In PCA: Data is projected onto principal components (eigenvectors):

$$\mathbf{x}_{\text{projected}} = \mathbf{U}_k \mathbf{U}_k^T \mathbf{x}$$

where $\mathbf{U}_k$ contains the top k eigenvectors.

Meaning: Projects high-dimensional data onto lower-dimensional subspace that captures maximum variance.

A14.

$$\mathbf{AB} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 4 & 6 \\ 1 & 3 \end{bmatrix}$$

$$\mathbf{BA} = \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 4 \\ 1 & 5 \end{bmatrix}$$

Note: $\mathbf{AB} \neq \mathbf{BA}$ (non-commutative)

A15. The **kernel (null space)** of T is:

$$\ker(T) = \{\mathbf{v} : T(\mathbf{v}) = \mathbf{0}\}$$

Geometric interpretation: All vectors that get "collapsed" to the zero vector.

Example: For projection onto a line, the kernel is all vectors perpendicular to that line.

ML relevance: In neural networks, if many inputs map to zero, the transformation loses information.

A16. $T(x, y) = (x + 2y, 3x - y)$

Matrix representation: Find where basis vectors map:

$$T(1, 0) = (1, 3)$$
$$T(0, 1) = (2, -1)$$

These form the columns:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & -1 \end{bmatrix}$$

Verification:

$$\begin{bmatrix} 1 & 2 \\ 3 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x + 2y \\ 3x - y \end{bmatrix} \checkmark$$

A17. Horizontal shear: $\mathbf{S} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$

$$\mathbf{S} \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 1 + 6 \\ 3 \end{bmatrix} = \begin{bmatrix} 7 \\ 3 \end{bmatrix}$$

Explanation: x-coordinate shifts by $k \times y = 2 \times 3 = 6$ units.

A18.

$$\mathbf{Wx} = \begin{bmatrix} 0.5 & -0.3 \\ 0.2 & 0.8 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.5 - 0.3 \\ 0.2 + 0.8 \end{bmatrix} = \begin{bmatrix} 0.2 \\ 1.0 \end{bmatrix}$$

Interpretation: In a neural network, this is the pre-activation output for input $\mathbf{x} = [1, 1]^T$.

A19. Solve using Gaussian elimination:

$$\begin{cases} x + 2y - z = 3 \\ 2x + y + z = 1 \\ x - y + z = -1 \end{cases}$$

Augmented matrix:

$$\left[\begin{array}{ccc|c} 1 & 2 & -1 & 3 \\ 2 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \end{array}\right]$$

Step 1: $R_2 \leftarrow R_2 - 2R_1$, $R_3 \leftarrow R_3 - R_1$:

$$\left[\begin{array}{ccc|c} 1 & 2 & -1 & 3 \\ 0 & -3 & 3 & -5 \\ 0 & -3 & 2 & -4 \end{array}\right]$$

Step 2: $R_3 \leftarrow R_3 - R_2$:

$$\left[\begin{array}{ccc|c} 1 & 2 & -1 & 3 \\ 0 & -3 & 3 & -5 \\ 0 & 0 & -1 & 1 \end{array}\right]$$

Back substitution:

$$-z = 1 \Rightarrow z = -1$$

$$-3y + 3(-1) = -5 \Rightarrow y = \frac{2}{3}$$

$$x + 2(\frac{2}{3}) - (-1) = 3 \Rightarrow x = \frac{4}{3}$$

Solution: $x = \frac{4}{3}, y = \frac{2}{3}, z = -1$

A20. Scaling by 2:

$$\mathbf{S} = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

Rotation by 45°:

$$\mathbf{R} = \begin{bmatrix} \cos 45 & -\sin 45 \\ \sin 45 & \cos 45 \end{bmatrix} = \begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{bmatrix}$$

Composition RS (scale then rotate):

$$\mathbf{RS} = \begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} \frac{2}{\sqrt{2}} & -\frac{2}{\sqrt{2}} \\ \frac{2}{\sqrt{2}} & \frac{2}{\sqrt{2}} \end{bmatrix} = \begin{bmatrix} \sqrt{2} & -\sqrt{2} \\ \sqrt{2} & \sqrt{2} \end{bmatrix}$$

7.3 Level 3 Solutions

A21. Proof of associativity:

Let $T_1 : \mathbb{R}^n \to \mathbb{R}^m$, $T_2 : \mathbb{R}^m \to \mathbb{R}^p$, $T_3 : \mathbb{R}^p \to \mathbb{R}^q$ with matrices $\mathbf{A}_1, \mathbf{A}_2, \mathbf{A}_3$.

Left side: $(T_3 \circ T_2) \circ T_1$

$$\begin{aligned} [(T_3 \circ T_2) \circ T_1](\mathbf{x}) &= (T_3 \circ T_2)(T_1(\mathbf{x})) \\ &= T_3(T_2(T_1(\mathbf{x}))) \\ &= \mathbf{A}_3(\mathbf{A}_2(\mathbf{A}_1\mathbf{x})) \\ &= (\mathbf{A}_3\mathbf{A}_2)\mathbf{A}_1\mathbf{x} \end{aligned}$$

Right side: $T_3 \circ (T_2 \circ T_1)$

$$\begin{aligned} [T_3 \circ (T_2 \circ T_1)](\mathbf{x}) &= T_3[(T_2 \circ T_1)(\mathbf{x})] \\ &= T_3(T_2(T_1(\mathbf{x}))) \\ &= \mathbf{A}_3(\mathbf{A}_2\mathbf{A}_1)\mathbf{x} \end{aligned}$$

By associativity of matrix multiplication: $(\mathbf{A}_3\mathbf{A}_2)\mathbf{A}_1 = \mathbf{A}_3(\mathbf{A}_2\mathbf{A}_1)$ ✓

A22. Without activation functions, a multi-layer network becomes:

$$\mathbf{y} = \mathbf{W}_L(\mathbf{W}_{L-1}(\cdots(\mathbf{W}_2(\mathbf{W}_1\mathbf{x}))\cdots))$$

By associativity:

$$\mathbf{y} = (\mathbf{W}_L\mathbf{W}_{L-1}\cdots\mathbf{W}_2\mathbf{W}_1)\mathbf{x} = \mathbf{W}_{\text{eff}}\mathbf{x}$$

Explanation: Multiple linear transformations compose to a single linear transformation $\mathbf{W}_{\text{eff}}$. This eliminates any benefit of depth—the network is equivalent to a single-layer linear model.

Why activations matter: Non-linear activations (ReLU, sigmoid, tanh) break this composition, allowing networks to learn complex, non-linear functions.

A23. In backpropagation, gradients flow backward through composed transformations using the chain rule:

Forward pass (layer i):

$$\mathbf{z}^{(i)} = \mathbf{W}^{(i)}\mathbf{a}^{(i-1)}, \quad \mathbf{a}^{(i)} = \sigma(\mathbf{z}^{(i)})$$

Backward pass:

$$\frac{\partial L}{\partial \mathbf{W}^{(i)}} = \frac{\partial L}{\partial \mathbf{z}^{(i)}} \cdot (\mathbf{a}^{(i-1)})^T$$

$$\frac{\partial L}{\partial \mathbf{a}^{(i-1)}} = (\mathbf{W}^{(i)})^T \cdot \frac{\partial L}{\partial \mathbf{z}^{(i)}}$$

Key role of matrix multiplication:

- Gradients are computed via matrix-vector products
- Weight gradients require outer products
- Gradient flows backward through transpose of weight matrices
- Composition of Jacobians follows matrix multiplication rules

A24. Computational complexity comparison:
(a) Gaussian Elimination:

- Time: $O(n^3)$ operations
- Space: $O(n^2)$ for augmented matrix
- *Best for*: Single system, direct solution

(b) Matrix Inverse:

- Time: $O(n^3)$ to compute $\mathbf{A}^{-1}$, then $O(n^2)$ for $\mathbf{A}^{-1}\mathbf{b}$
- Space: $O(n^2)$ to store $\mathbf{A}^{-1}$
- *Best for*: Multiple systems with same $\mathbf{A}$, different $\mathbf{b}$
- *Drawback*: Numerically unstable, rarely used in practice

(c) LU Decomposition:

- Time: $O(n^3)$ for decomposition, $O(n^2)$ per solve
- Space: $O(n^2)$ for $\mathbf{L}$ and $\mathbf{U}$
- *Best for*: Multiple systems with same $\mathbf{A}$, reusable decomposition
- *Advantage*: More numerically stable than inverse

Summary table:

Method	Initial Cost	Per Solve	Multiple $\mathbf{b}$
Gaussian	$O(n^3)$	—	$O(n^3)$ each
Inverse	$O(n^3)$	$O(n^2)$	Efficient
LU	$O(n^3)$	$O(n^2)$	Efficient + Stable

A25. Feature transformation strategy for linear separability:
Problem: Given data that's not linearly separable in original space, design transformation.
Strategy:

1. **Analyze data distribution**: Identify non-linear patterns
2. **Design transformation matrix $\mathbf{W}$**:

$$\mathbf{z} = \mathbf{W}\mathbf{x}$$

where $\mathbf{W}$ creates features that capture non-linear relationships

3. **Common transformations**:
   - Rotation to align with principal axes: PCA
   - Projection onto discriminative directions: LDA
   - Polynomial features via basis expansion
4. **Example - XOR problem**:
   Original features: $\mathbf{x} = [x_1, x_2]^T$
   Transformation:
   $$\mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$
   New features:
   $$\mathbf{z} = \begin{bmatrix} x_1 + x_2 \\ x_1 - x_2 \end{bmatrix}$$
   This makes XOR linearly separable!
5. **Verification with data**:

x_1	x_2	Label	z_1	z_2	Separable?
0	0	0	0	0	
0	1	1	1	-1	Yes
1	0	1	1	1	(by z_1)
1	1	0	2	0	

A26. 3D transformations in data augmentation:
Applications:

1. **Rotation augmentation** (simulate different viewpoints):
   $$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$
   Example: Rotate 3D point cloud of an object by 30°:
   $$\mathbf{R}_y(30) = \begin{bmatrix} 0.866 & 0 & 0.5 \\ 0 & 1 & 0 \\ -0.5 & 0 & 0.866 \end{bmatrix}$$
2. **Scaling augmentation** (simulate different sizes):
   $$\mathbf{S} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \end{bmatrix}$$
   Example: Scale by 1.2 uniformly:
   $$\mathbf{S} = \begin{bmatrix} 1.2 & 0 & 0 \\ 0 & 1.2 & 0 \\ 0 & 0 & 1.2 \end{bmatrix}$$

3. **Combined transformation**:

$$\mathbf{T}_{\text{aug}} = \mathbf{S} \cdot \mathbf{R}_y(30) \cdot \mathbf{R}_x(15)$$

This creates diverse training samples from a single 3D object!

Benefits:

- Increases dataset size artificially
- Improves model robustness
- Simulates real-world variations

A27. Determinant under composition:
Theorem: $\det(\mathbf{AB}) = \det(\mathbf{A}) \cdot \det(\mathbf{B})$
Proof:
The determinant represents the scaling factor of volumes. When applying transformations $\mathbf{A}$ and $\mathbf{B}$ in sequence:

- $\mathbf{B}$ scales volume by factor $\det(\mathbf{B})$
- $\mathbf{A}$ further scales by factor $\det(\mathbf{A})$
- Total scaling: $\det(\mathbf{A}) \cdot \det(\mathbf{B})$

Algebraic proof sketch:
Using properties of determinants and the definition of matrix multiplication, one can show:

$$\det(\mathbf{AB}) = \sum_{\sigma} \operatorname{sgn}(\sigma) \prod_{i} (\mathbf{AB})_{i,\sigma(i)}$$

Through careful manipulation of permutations and products, this equals:

$$\left(\sum_{\tau} \operatorname{sgn}(\tau) \prod_{i} a_{i,\tau(i)} \right) \cdot \left(\sum_{\rho} \operatorname{sgn}(\rho) \prod_{j} b_{j,\rho(j)} \right) = \det(\mathbf{A}) \cdot \det(\mathbf{B})$$

Example:

$$\mathbf{A} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$$

$$\det(\mathbf{A}) = 6, \quad \det(\mathbf{B}) = 1$$

$$\mathbf{AB} = \begin{bmatrix} 2 & 4 \\ 0 & 3 \end{bmatrix}, \quad \det(\mathbf{AB}) = 6 = 6 \cdot 1 \checkmark$$

A28. Proof of distributivity:
To prove: $\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}$
Proof:
Let $\mathbf{A} = [a_{ij}]$, $\mathbf{B} = [b_{jk}]$, $\mathbf{C} = [c_{jk}]$

Element (i, k) of left side:

$$\begin{aligned}[\mathbf{A}(\mathbf{B}+\mathbf{C})]_{ik} &= \sum_j a_{ij}(b_{jk}+c_{jk}) \\ &= \sum_j (a_{ij}b_{jk}+a_{ij}c_{jk}) \\ &= \sum_j a_{ij}b_{jk} + \sum_j a_{ij}c_{jk}\end{aligned}$$

Element (i, k) of right side:

$$\begin{aligned}[\mathbf{AB}+\mathbf{AC}]_{ik} &= [\mathbf{AB}]_{ik} + [\mathbf{AC}]_{ik} \\ &= \sum_j a_{ij}b_{jk} + \sum_j a_{ij}c_{jk}\end{aligned}$$

Since all elements are equal: $\mathbf{A}(\mathbf{B}+\mathbf{C}) = \mathbf{AB}+\mathbf{AC}$ ✓

A29. Eigenvalues and iterative methods:

For iterative methods (Jacobi, Gauss-Seidel, etc.) solving $\mathbf{Ax} = \mathbf{b}$, convergence depends on the eigenvalues of the iteration matrix.

General form:

$$\mathbf{x}^{(k+1)} = \mathbf{G}\mathbf{x}^{(k)} + \mathbf{c}$$

Convergence theorem: The method converges if and only if the spectral radius $\rho(\mathbf{G}) < 1$, where:

$$\rho(\mathbf{G}) = \max_i |\lambda_i|$$

Interpretation:

- If all $|\lambda_i| < 1$: Errors decay exponentially → convergence
- If any $|\lambda_i| \geq 1$: Errors grow or persist → divergence
- Smaller $\rho(\mathbf{G})$: Faster convergence

Example - Jacobi method:
For $\mathbf{A} = \mathbf{D}+\mathbf{L}+\mathbf{U}$:

$$\mathbf{G}_{\text{Jacobi}} = -\mathbf{D}^{-1}(\mathbf{L}+\mathbf{U})$$

Convergence guaranteed if $\mathbf{A}$ is diagonally dominant.

A30. Determining system types:

Definitions:

- **Exactly determined**: $m = n$ (same number of equations and unknowns)
- **Over-determined**: $m > n$ (more equations than unknowns)
- **Under-determined**: $m < n$ (fewer equations than unknowns)

Determination criteria:
For system $\mathbf{Ax} = \mathbf{b}$ where $\mathbf{A} \in \mathbb{R}^{m\times n}$:

Type	Condition	Solution
Exactly determined	$m = n, \text{rank}(\mathbf{A}) = n$	Unique solution
Over-determined	$m > n$	No exact solution (use least squares)
Under-determined	$m < n$	Infinitely many solutions

ML implications:

1. **Over-determined** ($m > n$):
   - Common in ML: more data points than features
   - Use least squares: minimize $\|\mathbf{Ax} - \mathbf{b}\|^2$
   - Example: Linear regression with 1000 samples, 10 features
2. **Under-determined** ($m < n$):
   - More features than data points
   - Infinite solutions → need regularization
   - Example: 10 samples, 100 features (overfitting risk)
   - Use Ridge/Lasso to select unique solution
3. **Exactly determined**:
   - Rare in practice
   - Unique solution if $\mathbf{A}$ is invertible
   - Can still overfit if not representative

7.4 Multiple Choice Solutions

A31. (b) A is invertible

Explanation: For unique solution to exist, $\mathbf{A}$ must be square ($m = n$) AND have full rank (invertible). Then $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$ is the unique solution.

A32. (b) The network reduces to a single linear transformation

Explanation: Without non-linear activations, $\mathbf{y} = \mathbf{W}_L \cdots \mathbf{W}_1\mathbf{x} = \mathbf{W}_{\text{eff}}\mathbf{x}$, collapsing to a single linear layer. This eliminates any advantage of depth.

A33. (b) rank(A) + nullity(A) = number of columns

Explanation: For $\mathbf{A} \in \mathbb{R}^{m \times n}$:

$$\text{rank}(\mathbf{A}) + \dim(\ker(\mathbf{A})) = n$$

This fundamental theorem relates the dimension of the image and kernel.

A34. (c) Angles

Explanation: Linear transformations preserve:

- Lines → lines
- Parallel lines → parallel lines
- Ratios along lines

But generally do NOT preserve:

- Angles (except for rotations/reflections)
- Distances (except for isometries)

Example: Shear transformation distorts angles but preserves parallelism.
A35. (a) $O(mnp)$
Explanation: Computing $\mathbf{C} = \mathbf{AB}$ where $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{B} \in \mathbb{R}^{n \times p}$:

- $\mathbf{C}$ has $m \times p$ elements
- Each element requires n multiplications
- Total: mnp operations

8 Practice Problems

8.1 Computational Problems

1. **Calculate**: Solve the system
$$\begin{cases} 3x - y + 2z = 12 \\ x + 2y - z = -3 \\ 2x + y + 3z = 8 \end{cases}$$
2. **If** $\mathbf{A} = \begin{bmatrix} 1 & 3 & 2 \\ 0 & 1 & 4 \end{bmatrix}$, find the LU decomposition.
3. **Project** vector $\mathbf{v} = [2, 3, 1]^T$ onto plane spanned by $\mathbf{u}_1 = [1, 0, 0]^T$ and $\mathbf{u}_2 = [0, 1, 0]^T$.
4. **Calculate** composition of:
   - Rotation 60° about z-axis
   - Scaling by $(2, 2, 0.5)$
   - Reflection across xy-plane
5. **Apply** Gaussian elimination to:
$$\begin{bmatrix} 2 & 1 & -1 \\ -1 & 3 & 2 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$$

8.2 Conceptual Problems

6. **Why can't** we add a vector to a scalar in the equation $\mathbf{Ax} + c = \mathbf{b}$ without broadcasting?
7. **How does** composition of transformations relate to function composition in calculus?
8. **Explain** why the order of transformations matters geometrically.
9. **Why does** backpropagation use transpose of weight matrices?
10. **How do** linear transformations in neural networks create feature hierarchies?

8.3 Application Problems

11. **Design** a 3-layer neural network for XOR problem. Specify weight matrices and show forward pass.
12. **Implement** least squares solution for linear regression with 5 data points.
13. **Create** a data augmentation pipeline using 3D transformations for point clouds.
14. **Calculate** gradient update for 2-layer network with specific inputs and targets.
15. **Derive** the normal equations for ridge regression starting from the regularized loss function.